



Facultad de Ingeniería

Magíster en Ciencia de Datos e Inteligencia Artificial

**SOBREDURACIÓN EN LA TITULACIÓN DE LA EDUCACIÓN SUPERIOR
CHILENA (2020-2025)**

Informe final integrador — Fase 4: análisis, reproducibilidad y comunicación de
resultados

Informe final presentado para la asignatura Programación para la Ciencia de Datos
(202682.1927)

Autor:

Sebastian Antunez Noguera

Profesor:

Omar Salinas

Santiago, Chile

4 de octubre de 2026

Ficha del trabajo

Antecedente	Detalle
Asignatura	Programación para la Ciencia de Datos (202682.1927)
Programa	Magíster en Ciencia de Datos e Inteligencia Artificial
Profesor	Omar Salinas
Autor	Sebastian Antunez Noguera (trabajo individual)
Entrega	Sumativa 3 — Proyecto final integrador, Fase 4
Repositorio	https://github.com/santunez75/proyecto-titulados
Notebooks	F1, F2 (tres), F3 y F4: seis notebooks ejecutados de corrido
Fuente de datos	SIES, Titulados de Educación Superior 2020-2025 (6 archivos CSV, 955 MB)
Conjunto analítico	1.246.760 titulados de pregrado, 47 variables
Entorno	Python 3.14.7 · pandas 3.0.5 · NumPy 2.5.3 · Windows 11
Pruebas automatizadas	73 (22 del pipeline de F2 + 51 del núcleo de F3), todas aprobadas
Reglas de validación	9, todas aprobadas

Todas las cifras, tablas y figuras de este informe fueron generadas por los notebooks del repositorio y son reproducibles ejecutándolos. Sus salidas quedan en reports/tables/ y reports/figures/, y cada afirmación puede rastrearse hasta el archivo que la respalda.

Índice

Para actualizar el índice en Word: clic en la tabla y F9, o clic derecho → Actualizar campos.

1. Introducción y contextualización	5
2. Definición de la problemática y objetivos	6
2.1 Problemática y pregunta técnica	6
2.2 Objetivos	6
2.3 Alcance, restricciones y supuestos.....	7
3. Metodología del desarrollo técnico.....	9
3.1 Herramientas del ecosistema científico de Python	9
3.2 Reproducibilidad técnica	9
3.3 Control de versiones	10
3.4 Fase 1 — Definición del problema y entorno	11
3.5 Fase 2 — Preprocesamiento y transformación del dataset.....	11
La transformación central: reconstruir la duración real.....	12
3.6 Fase 3 — Núcleo algorítmico, eficiencia y orientación a objetos	13
Algoritmos implementados y verificados	13
Mediciones de eficiencia.....	13
Programación orientada a objetos	15
3.7 Arquitectura del código.....	16
3.8 Validación técnica y verificación del código	17
Nivel 1: corrección contra referencias independientes	17
Nivel 2: pruebas automatizadas.....	17
Nivel 3: reglas de coherencia del dominio.....	18
Nivel 4: ejecución de extremo a extremo	18
4. Resultados.....	20
4.1 Panorama general.....	20
4.2 Distribución de la sobreduración	20
4.3 Factores asociados: área, institución y género.....	21
4.4 Modalidad y territorio	22
4.5 Evolución temporal: el efecto de la pandemia	23
4.6 Sesgo de truncamiento por cohorte.....	24
4.7 Modelación desde primeros principios.....	25
5. Discusión	27
5.1 Contraste con las hipótesis iniciales	27
5.2 Limitaciones y riesgos	27
5.3 Relación entre las fases y decisiones determinantes	28

6. Conclusiones	29
6.1 Hallazgos sustantivos	29
6.2 Aprendizajes técnicos	29
6.3 Mejoras propuestas, basadas en la evidencia del proyecto	30
6.4 Comunicación de resultados	30
7. Trazabilidad de mejoras	32
8. Repositorio y notebooks	33
8.1 Estructura del repositorio	33
8.2 Notebooks del proyecto	33
9. Referencias	35
Bibliografía docente	35
Fuente de datos	35
Documentación técnica oficial	35
Bibliografía complementaria	35
Anexos	37
Anexo A. Salida completa de las pruebas automatizadas	37
Anexo B. Historial completo de commits	38
Anexo C. Contribuciones y ramas	39

1. Introducción y contextualización

El sistema de educación superior chileno declara, para cada carrera y cada institución, una duración teórica: el número de semestres que el plan de estudios compromete con el estudiante. Esa cifra orienta decisiones concretas —cuánto financiamiento solicitar, cuántos años de arancel prever, cuándo se espera entrar al mercado laboral— y es la base sobre la que se calculan la duración de las becas y los créditos estatales.

El sistema no publica, en cambio, cuánto tardan efectivamente los estudiantes en titularse. La diferencia entre ambas magnitudes, que este proyecto denomina sobreduración, no existe como variable en ningún registro oficial. Su ausencia no es menor: cada semestre adicional representa arancel pagado, financiamiento consumido e ingresos laborales postergados, tanto para la familia como para el Estado que subsidia la matrícula.

El Servicio de Información de Educación Superior (SIES) publica anualmente la base de Titulados de Educación Superior, con el detalle de cada proceso de titulación del país. Esa base contiene el año y semestre de ingreso a la carrera de origen, la fecha de obtención del título y la duración declarada del plan. Con esos tres elementos la sobreduración puede reconstruirse: es lo que hace este proyecto.

La relevancia del problema es doble. Para la política pública, dimensionar el rezago y localizar dónde se concentra permite focalizar programas de acompañamiento y revisar planes de estudio cuya duración declarada resulta sistemáticamente irrealizable. Para el estudiante y su familia, es la diferencia entre una expectativa informada y una proyección que la evidencia contradice.

Este informe cierra un proyecto desarrollado en cuatro fases, cada una construida sobre la anterior. La Fase 1 definió el problema y levantó un entorno reproducible. La Fase 2 construyó el pipeline que convierte 955 MB de archivos crudos en un conjunto analítico validado. La Fase 3 implementó desde primeros principios el núcleo algorítmico —algoritmos, mediciones de complejidad, modelo de dominio orientado a objetos y modelos predictivos—. La Fase 4, que aquí se documenta, ejecuta ese instrumental de extremo a extremo, produce los resultados finales, los discute y los comunica.

El propósito general del trabajo es, por tanto, doble: responder con evidencia a una pregunta sustantiva sobre el sistema educativo chileno y hacerlo con un desarrollo técnico íntegramente reproducible, trazable y verificable por terceros.

2. Definición de la problemática y objetivos

2.1 Problemática y pregunta técnica

El hallazgo que delimitó el proyecto apareció al leer el esquema de registro oficial del SIES. Las tres columnas que parecían contener la respuesta —`dur_estudio_carr`, `dur_proceso_tit` y `dur_total_carr`— no registran lo que el estudiante tardó, sino lo que el plan declara. Son un atributo de la carrera, no del estudiante. La duración real no está en la base.

Pregunta técnica que guía el proyecto. ¿Es posible reconstruir la duración real de la titulación a partir de los registros administrativos disponibles y, con esa variable derivada, caracterizar la magnitud y la distribución del rezago en la educación superior chilena?

La pregunta es técnica antes que sustantiva: su respuesta depende de que la reconstrucción sea correcta, verificable y defendible. De ahí que buena parte del esfuerzo del proyecto se haya destinado a validar esa variable y no solo a describirla.

2.2 Objetivos

Objetivo general. Construir, validar y analizar un indicador de sobreduración en la titulación de pregrado en Chile a partir de los registros del SIES 2020-2025, mediante un desarrollo en Python íntegramente reproducible y versionado.

Objetivos específicos:

1. Consolidar los seis archivos anuales del SIES en un único conjunto analítico, resolviendo los códigos centinela, los duplicados y las inconsistencias de codificación documentadas por la fuente.
2. Reconstruir la duración real de cada trayectoria y derivar de ella la sobreduración, el índice de duración y una categorización del rezago, dejando registro auditable de cada decisión.
3. Implementar desde primeros principios los algoritmos centrales del análisis, medir su complejidad temporal y espacial con métodos formales y justificar la elección final frente a las implementaciones de biblioteca.
4. Organizar el dominio con programación orientada a objetos, aplicando encapsulamiento, herencia, polimorfismo y patrones de diseño donde resuelvan un problema concreto.

5. Caracterizar el rezago por área de conocimiento, tipo de institución, género, modalidad, territorio y cohorte, y comunicar los resultados mediante visualizaciones analíticas.
6. Garantizar la reproducibilidad del proyecto completo mediante entorno virtual, dependencias fijadas, pruebas automatizadas y control de versiones con historial verificable.

2.3 Alcance, restricciones y supuestos

Tabla 1. Delimitación del estudio

Dimensión	Definición adoptada	Fundamento
Población	Titulados de pregrado del sistema chileno	Posgrado y postítulo no son comparables en duración teórica
Período	Procesos de titulación 2020-2025 (más el parcial 2026-S1)	Cobertura completa de los archivos publicados por el SIES
Unidad de análisis	El proceso de titulación, no la persona	Un estudiante con dos títulos aparece dos veces, por diseño de la fuente
Variable objetivo	$\text{sobreduracion_sem} = \text{duración real} - \text{duración teórica}$	Derivada; no existe en la fuente
Fuera de alcance	Deserción, rendimiento académico, costo monetario	La base contiene únicamente a quienes se titularon

Restricciones:

- La base registra únicamente titulados. Quien abandonó no aparece, de modo que el rezago medido es el de los que llegaron a egresar.
- El semestre de titulación se deriva del mes: hasta julio se asigna al primer semestre, de agosto en adelante al segundo. La fuente no entrega el semestre de forma directa.
- Las duraciones teóricas las reporta cada institución y no están auditadas centralmente.
- Los datos crudos (955 MB) no se versionan en el repositorio por límites de tamaño; se distribuye una muestra de 5.000 registros que permite ejecutar todo el flujo.

Supuestos:

- La trayectoria entre el ingreso y la titulación se supone continua. La fuente no informa congelamientos, de modo que un semestre suspendido se contabiliza como tiempo transcurrido.
- El año y semestre de ingreso a la carrera de origen identifican correctamente el inicio de la trayectoria conducente al título obtenido.
- La duración teórica declarada es la vigente para la cohorte del estudiante; los cambios de malla dentro del período no son observables.

3. Metodología del desarrollo técnico

Esta sección describe cómo se construyó el proyecto: las herramientas empleadas, las decisiones técnicas de cada fase y la forma en que se garantizó que los resultados puedan reproducirse y auditarse.

3.1 Herramientas del ecosistema científico de Python

Tabla 2. Herramientas utilizadas y su función en el proyecto

Herramienta	Versión	Función en el proyecto
pandas	3.0.5	Lectura por bloques de los CSV, tipos anulables, agregaciones y tablas de resultados
NumPy	2.5.3	Álgebra de matrices en los modelos, operaciones vectorizadas, ajuste log-log de las curvas de escalamiento
PyArrow	25.0.1	Motor Parquet: el consolidado de 1,7 millones de filas se escribe y relee en segundos
Matplotlib / seaborn	3.11.1 / 0.13.2	Todas las figuras, con estilo unificado definido en src/viz.py
JupyterLab	4.6.3	Entorno de los seis notebooks del proyecto
nbconvert	7.17.1	Ejecución de los notebooks de corrido, sin intervención manual
pytest	9.1.1	73 pruebas automatizadas de casos normales, límite y excepciones
timeit / tracemalloc	estándar	Mediciones formales de tiempo y de pico de memoria
Git y GitHub	—	Control de versiones, ramas por fase y trazabilidad del desarrollo

La integración entre herramientas es coherente de extremo a extremo: los notebooks importan el paquete src/, que a su vez usa pandas y NumPy; las figuras y tablas que producen se escriben en reports/ y desde ahí las lee este informe. Ningún número de este documento se transcribió a mano.

3.2 Reproducibilidad técnica

El proyecto se ejecuta en un entorno virtual con dependencias fijadas por versión exacta. Cualquier equipo con Python 3.14 reproduce el análisis con cuatro comandos.

Instalación y ejecución completas del proyecto

```
git clone https://github.com/santunez75/proyecto-titulados.git
cd proyecto-titulados
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pytest tests/ -q      # 73 pruebas, sin datos externos
.venv\Scripts\python.exe -m src.pipeline          # regenera el dataset (requiere los CSV)
.venv\Scripts\python.exe -m jupyter lab          # ejecuta los notebooks
```

Tres decisiones sostienen esa reproducibilidad, y las tres se adoptaron después de detectar un fallo concreto al reproducir el proyecto en un segundo equipo:

1. Versiones fijadas con == en requirements.txt. El notebook de la Fase 4 compara las versiones instaladas contra las declaradas y reporta cuántas coinciden: si alguna difiere, quien ejecute el proyecto lo sabe antes de interpretar los resultados.
2. Degradación controlada de la carga de datos. Si el Parquet completo no existe, cargar_datos() usa la muestra versionada de 5.000 registros y lo informa por pantalla. El notebook corre en un equipo sin los CSV originales, en lugar de fallar.
3. Pruebas ejecutadas con el intérprete del entorno virtual, no con el del kernel. Un kernel de Jupyter distinto al .venv hacía fallar la celda de pruebas sin mostrar la causa, porque el error viajaba por stderr. Ahora la celda apunta al intérprete correcto e imprime stderr cuando el código de salida no es cero.

Las tres comparten un criterio que atraviesa el proyecto: convertir un fallo silencioso en un comportamiento explícito. Un error que se ve es un error que se corrige; uno que no se ve contamina resultados.

3.3 Control de versiones

El repositorio acumula 36 commits distribuidos a lo largo de las cuatro fases. Los mensajes siguen la convención de commits semánticos (feat, fix, perf, test, docs, chore) y explican qué se hizo y por qué, no solo qué archivo cambió.

El trabajo de las Fases 2 y 3 se desarrolló en ramas propias —fase-2/pipeline-datos y fase-3/nucleo-algoritmico— que se integraron a main mediante fusiones explícitas sin avance rápido, de modo que el punto de integración queda registrado en el historial y la estructura del desarrollo puede reconstruirse.

Tabla 3. Commits más recientes del repositorio (36 en total)

Hash	Fecha	Mensaje
12ae178	2026-09-27	feat(f4): notebook del reporte analítico integrador con cuatro figuras
7538f5c	2026-09-24	chore(f3): re-ejecuta los notebooks en el entorno local y actualiza salidas
b5523ad	2026-09-24	docs(f3): versión final del informe
b62d36a	2026-09-21	docs(f3): informe y evidencias alineados con el historial definitivo
9062995	2026-09-21	merge: integra la rama fase-3/nucleo-algoritmico en main
58cb930	2026-09-19	docs(f3): actualiza el informe y las evidencias con el historial definitivo

Hash	Fecha	Mensaje
b769379	2026-09-19	docs(f3): agrega el PDF del informe con la fecha de entrega
d10b470	2026-09-19	docs(f3): Primer Informe F3
6d348f8	2026-09-19	docs(f3): informe tecnico de la Fase 3 y evidencias de ejecucion
b8770f5	2026-09-19	docs(f3): notebook del nucleo algoritmico ejecutado de corrido
d6df25e	2026-09-19	test(nucleo): 51 pruebas de casos normales, limite y excepciones
40be4d9	2026-09-19	feat(nucleo): preparacion, modelos por descenso de gradiente y fachada

git log --oneline --graph: ramas y puntos de integración

* 12ae178 feat(f4): notebook del reporte analitico integrador con cuatro figuras
* 7538f5c chore(f3): re-ejecuta los notebooks en el entorno local y actualiza salidas
* b5523ad docs(f3): versión final del informe
* b62d36a docs(f3): informe y evidencias alineados con el historial definitivo
* 9062995 merge: integra la rama fase-3/nucleo-algoritmico en main
\
* 58cb930 docs(f3): actualiza el informe y las evidencias con el historial definitivo
* b769379 docs(f3): agrega el PDF del informe con la fecha de entrega
* d10b470 docs(f3): Primer Informe F3
* 6d348f8 docs(f3): informe tecnico de la Fase 3 y evidencias de ejecucion
* b8770f5 docs(f3): notebook del nucleo algoritmico ejecutado de corrido
* d6df25e test(nucleo): 51 pruebas de casos normales, limite y excepciones

Contribuciones. El proyecto es de autoría individual: la totalidad de los commits corresponde al autor. El historial registra dos identidades de Git, correspondientes a los dos equipos utilizados durante el desarrollo, ambas vinculadas a la cuenta institucional. El detalle está en el Anexo C.

3.4 Fase 1 — Definición del problema y entorno

La Fase 1 estableció tres cosas que condicionaron todo lo demás: la pregunta técnica, el entorno reproducible y la organización modular del código. La decisión de separar el código en módulos con una responsabilidad cada uno, en lugar de escribirlo dentro de los notebooks, permitió que el proyecto creciera de 5 a 16 módulos sin reescrituras.

3.5 Fase 2 — Preprocesamiento y transformación del dataset

El pipeline de la Fase 2 convierte seis CSV heterogéneos en un conjunto analítico validado. Se ejecuta con un comando y deja bitácora de cada paso. La tabla documenta las ocho etapas, su efecto cuantificado y la justificación técnica de cada una.

Tabla 4. Pipeline de preprocesamiento: tratamiento, efecto y justificación

Paso	Tratamiento	Efecto	Justificación técnica
Ingesta	Lectura por bloques de 6 CSV con dtypes declarados y validación de esquema	1.710.167 filas	Declarar los tipos evita inferencias erróneas y reduce el uso de memoria a la mitad
Códigos centinela	9995/9998/9999/1900, 19000101 y semestre 0 convertidos a nulos explícitos	62.492 en año de ingreso	Procesados como números producirían duraciones imposibles; hacerlos visibles precede a cualquier decisión
Normalización de texto	18 columnas categóricas homogeneizadas sobre su catálogo de valores únicos	8,6 millones de celdas	Evita categorías duplicadas por capitalización; operar sobre el catálogo reduce el costo de 1,7 millones de cadenas a unos miles
Restricción a pregrado	Filtro nivel_global == Pregrado	398.391 excluidos	Posgrado y postítulo no son comparables en duración teórica
Duplicados	Clave estudiante + carrera + fecha de titulación; filas sin identificador fuera del cotejo	72 eliminados	Pandas trata dos nulos como iguales; incluirlos colapsaría estudiantes distintos
Descarte sin insumos	Filas sin año o semestre de ingreso utilizable	61.825 (4,7 %)	Imputar el inicio de la trayectoria equivaldría a inventar la variable objetivo
Derivación y casting	Duración real, sobreduración, índice, edad y categoría de rezago en enteros anulables	1.246.760 finales	Enteros exactos en lugar de coma flotante: sin errores de redondeo en las comparaciones
Atípicos	Fuera de [1, 40] semestres o edad fuera de [15, 90], marcados con un indicador	3.119 (0,25 %)	Umbrales parametrizados en config.py; marcar en vez de eliminar deja la decisión visible y reversible

Retención final: 1.246.760 de 1.710.167 filas, un 72,9 %. Cada exclusión está justificada y cuantificada en la bitácora de limpieza que el pipeline escribe en `reports/tables/bitacora_limpieza.csv`.

La transformación central: reconstruir la duración real

`src/transformacion.py` — la derivación sobre la que descansa todo el análisis

```
# La duracion real no existe en la fuente: se deriva del ingreso y la titulacion.
# El semestre se infiere del mes porque la base no lo entrega directamente.
semestre_titulo = np.where(mes_titulo <= 7, 1, 2)

duracion_real_sem = (anio_titulo - anio_ing_carr_ori) * 2 \
    + (semestre_titulo - sem_ing_carr_ori) + 1

sobreduracion_sem = duracion_real_sem - dur_total_carr # variable objetivo
```

```
indice_duracion = duracion_real_sem / dur_total_carr # razon, comparable entre carreras
```

El «+1» no es un ajuste arbitrario: un estudiante que ingresa y se titula en el mismo semestre cursó un semestre, no cero. La coherencia aritmética de esta derivación se comprueba como regla de validación sobre el dataset completo.

3.6 Fase 3 — Núcleo algorítmico, eficiencia y orientación a objetos

La Fase 2 resolvió el problema con pandas. La Fase 3 preguntó qué hay debajo: implementó desde primeros principios los algoritmos centrales del análisis, midió su costo y organizó el dominio con programación orientada a objetos.

Algoritmos implementados y verificados

src/nucleo/algoritmos.py — agregación iterativa y selección recursiva

```
def agrupar_media(claves, valores):
    """Media por clave en una sola pasada. Tiempo O(n), espacio O(k)."""
    if len(claves) != len(valores):
        raise ValueError("claves y valores deben tener el mismo largo")
    suma, cuenta = {}, {} # dos acumuladores por grupo
    for clave, valor in zip(claves, valores): # una unica pasada sobre los datos
        suma[clave] = suma.get(clave, 0.0) + valor
        cuenta[clave] = cuenta.get(clave, 0) + 1
    return {clave: suma[clave] / cuenta[clave] for clave in suma}

def _quickselect(datos, k, rng):
    if len(datos) == 1: # caso base
        return datos[0]
    pivote = datos[rng.randrange(len(datos))] # pivote aleatorio: O(n) esperado
    menores = [x for x in datos if x < pivote] # tres particiones: la variable
    iguales = [x for x in datos if x == pivote] # tiene miles de valores repetidos
    mayores = [x for x in datos if x > pivote]
    if k < len(menores): # recurre solo donde esta la posicion
        return _quickselect(menores, k, rng)
    if k < len(menores) + len(iguales):
        return pivote
    return _quickselect(mayores, k - len(menores) - len(iguales), rng)
```

Cada algoritmo se verificó contra una referencia independiente sobre los datos reales antes de medir su eficiencia: la agregación contra groupby de pandas, el ordenamiento contra sorted, la mediana y el percentil contra NumPy, y el conteo por umbral contra value_counts. Medir un algoritmo incorrecto no tiene sentido.

Mediciones de eficiencia

Las mediciones usan timeit.repeat y reportan el mínimo de las repeticiones: el ruido del sistema operativo solo puede sumar tiempo, de modo que el mínimo es el mejor estimador del costo intrínseco (Python Software Foundation, 2025a). La memoria se mide con

tracemalloc como pico de asignación (Python Software Foundation, 2025b). Además del tamaño fijo, cada algoritmo se mide para tamaños crecientes y se ajusta una recta en escala log-log cuya pendiente es el exponente empírico de crecimiento (Cormen et al., 2022).

Tabla 5. Tiempo y memoria por implementación ($n = 200.000$ elementos)

Operación	Implementación	Tiempo	Memoria (KB)	Razón
agregación	agrupar_media $O(n)$	0,0480 s	0,7	5,7×
agregación	agrupar_media_ingenua $O(n \cdot k)$	0,0860 s	705,5	10,2×
agregación	pandas groupby	0,0084 s	3132,6	1,0×
ordenamiento	numpy.sort	0,0087 s	3127,8	1,0×
ordenamiento	sorted (Timsort en C)	0,0119 s	2343,6	1,4×
ordenamiento	ordenar_merge $O(n \log n)$	0,5891 s	3250,9	68,1×
mediana	numpy.median	0,0094 s	3128,2	1,0×
mediana	mediana_ordenando $O(n \log n)$	0,0121 s	2343,6	1,3×
mediana	quickselect $O(n)$	0,0528 s	4422,2	5,6×
búsqueda	busqueda_binaria $O(\log n)$	1,0 μ s	0,0	1,0×
búsqueda	busqueda_lineal $O(n)$	0,0063 s	0,1	6344,0×

La razón indica cuántas veces más lenta es cada implementación respecto de la mejor de su operación. La inserción $O(n^2)$ se midió aparte con $n = 3.000$.

Tabla 6. Cota teórica frente a exponente empírico

Algoritmo	Cota teórica	Exponente empírico	Rango de n medido
agrupar_media	$O(n)$	0,96	20.000 – 640.000
ordenar_merge	$O(n \log n)$	1,08	20.000 – 640.000
quickselect	$O(n)$	1,22	20.000 – 640.000
sorted	(C)	1,03	20.000 – 640.000
ordenar_insercion	$O(n^2)$	2,00	500 – 4.000

Las decisiones de optimización derivadas de estas mediciones son cuatro, y todas están implementadas en el código:

1. La agregación ingenua $O(n \cdot k)$ se reemplazó por una de una sola pasada $O(n)$: 1,9 veces más rápida con diez grupos y 860 veces más económica en memoria (0,7 KB frente a 603,9 KB), porque no materializa una sublista por grupo.

2. Los conteos por umbral se resuelven con dos búsquedas binarias recursivas sobre una copia ordenada: $O(\log n)$ por consulta, 6.516 veces más rápido que el recorrido lineal.
3. Los totales del árbol jerárquico se memorizan, lo que evita recorrer el subárbol en cada consulta.
4. pandas y NumPy se conservan para las operaciones masivas. A igual complejidad su constante es uno o dos órdenes de magnitud menor por estar escritos en C: sorted y numpy.sort superan a merge sort entre 48 y 66 veces. Los algoritmos propios se reservan para los casos en que su orden es estrictamente mejor y no existe equivalente en C sobre objetos.

Una excepción resultó instructiva: quickselect es $O(n)$ en número de comparaciones, pero su exponente empírico es 1,22 porque cada nivel de recursión asigna tres listas nuevas. La cota describe el crecimiento, no la constante ni el comportamiento de la memoria. Ese contraste entre teoría y medición es, en sí mismo, un resultado del proyecto.

Programación orientada a objetos

Las clases del núcleo se diseñaron a partir de una necesidad concreta cada una, y no al revés: encapsulamiento donde hay invariantes que proteger, herencia donde hay comportamiento común real, composición donde la relación es de pertenencia.

src/nucleo/ — encapsulamiento, polimorfismo y recursión sobre objetos

class Titulado:	
__slots__ = ("_mrun", "_duracion_teorica", "_duracion_real", ...) # sin __dict__	
@duracion_real.setter	
def duracion_real(self, valor: int) -> None:	
if int(valor) < config.DURACION_REAL_MIN: # el invariante vive en el objeto	
raise ValueError("la duracion real debe ser >= 1")	
self._duracion_real = int(valor)	
@property	
def sobreduracion(self) -> int: # derivada: nunca inconsistente	
return self._duracion_real - self._duracion_teorica	
class Metrica(ABC):	# patron Strategy
@abstractmethod	
def calcular(self, cohorte) -> float: ...	# cada subclase aporta la formula
class NodoJerarquia:	# patron Composite
def total(self) -> int: # recursivo y memorizado	
if self._cache_total is None:	

```

self._cache_total = len(self._titulados) + sum(h.total() for h in
self._hijos.values())
return self._cache_total

```

Tabla 7. Patrones de diseño aplicados, función y alternativa descartada

Patrón	Dónde	Función en la arquitectura	Alternativa descartada
Strategy	Metrica y subclases	Indicadores intercambiables sin tocar Cohorte ni la jerarquía	Un if/elif por métrica dentro de Cohorte
Composite	NodoJerarquia	Misma interfaz en hojas y nodos internos; agregación recursiva uniforme	Un groupby distinto por nivel: tres códigos para la misma pregunta
Template Method	ModeloGradiente.ajustar	El bucle de optimización se escribe una vez; cada modelo aporta enlace y pérdida	Copiar el bucle en cada modelo: duplicación y divergencia futura
Facade	NucleoAnalitico	Los notebooks no dependen de los detalles internos de nueve módulos	Que el notebook coordine cada módulo: alto acoplamiento
Composición	PreparadorModelacion	Tiene un escalador y un codificador; los coordina sin heredar de ellos	Herencia múltiple: relación «es un» incorrecta

3.7 Arquitectura del código

El proyecto se organiza en dieciséis módulos agrupados por responsabilidad. La arquitectura se rige por tres criterios verificables en el código, no por convenciones declaradas.

Arquitectura del paquete y sentido de las dependencias

src/	
-- config.py	Rutas, esquema de datos y parametros (unica fuente de constantes)
-- ingesta.py	Lectura por bloques de los CSV y consolidacion en Parquet
-- limpieza.py	Centinelas, faltantes, duplicados y bitacora auditable
-- transformacion.py	Duracion real, sobreduracion, indice y categorias de rezago
-- validacion.py	Motor de reglas (Regla, ValidadorDataset) reutilizado en F3 y F4
-- viz.py	Estilo unificado de figuras y formato numerico chileno
-- pipeline.py	Orquestador ejecutable de F1-F2
\-- nucleo/	Nucleo algoritmico de la Fase 3
-- algoritmos.py	Agregacion, merge sort, quickselect, busqueda binaria
-- benchmark.py	timeit + tracemalloc, curvas, exponente empirico
-- dominio.py	Titulado y Cohorte (encapsulamiento, protocolo de secuencia)
-- metricas.py	Metrica abstracta y seis subclases (Strategy)
-- jerarquia.py	NodoJerarquia recursivo con memorizacion (Composite)
-- preparacion.py	Transformador -> escaladores y one-hot (herencia)
-- modelos.py	Regresion lineal y logistica por gradiente (Template Method)
-- fachada.py	NucleoAnalitico (Facade): punto de entrada unico
\-- validacion_f3.py	Reglas de coherencia sobre el motor de la Fase 2

dependencias: fachada -> {dominio, metricas, jerarquia, preparacion, modelos} -> algoritmos
benchmark y algoritmos no importan nada del resto del proyecto

- **Alta cohesión.** Cada módulo agrupa lo que cambia junto: los algoritmos no saben qué es un titulado; las métricas no saben cómo se construye una cohorte; los modelos no saben de dónde salen las matrices.
- **Bajo acoplamiento.** Las dependencias apuntan en un solo sentido y los módulos se comunican por interfaces pequeñas: una secuencia iterable, una matriz NumPy, una función que devuelve un booleano y un detalle. Ese bajo acoplamiento es lo que permitió probar cada módulo por separado con datos construidos en memoria, sin necesidad de los archivos del SIES.
- **Extensibilidad.** Agregar un nivel a la jerarquía es cambiar una tupla de nombres; agregar un modelo es escribir dos ganchos; agregar una regla de validación es una línea. Ninguna de esas extensiones exige modificar código existente.

La recursividad se aplica donde el problema es recursivo por naturaleza y no como ejercicio: en el ordenamiento por mezcla, en la selección del k-ésimo menor, en las búsquedas binarias y en el árbol jerárquico, donde insertar, contar y recorrer se definen sobre un nodo en términos de sus hijos. En todos los casos la profundidad está acotada por $O(\log n)$ —21 niveles para 1,25 millones de elementos— y el caso base está explícito en las primeras líneas.

3.8 Validación técnica y verificación del código

La verificación opera en cuatro niveles, todos con evidencia registrada en el repositorio.

Nivel 1: corrección contra referencias independientes

Cada algoritmo propio se contrasta con una implementación externa sobre los datos reales. Las comparaciones son aserciones dentro de los notebooks: una discrepancia detiene la ejecución en lugar de producir un resultado silenciosamente incorrecto.

Nivel 2: pruebas automatizadas

Tabla 8. Cobertura de la suite de pruebas

Módulo	Casos normales	Casos límite	Excepciones
pipeline (F2)	Centinelas, duplicados, derivación, reglas de validación	Dataset de una fila; columnas ausentes	Esquema inválido; rutas inexistentes

Módulo	Casos normales	Casos límite	Excepciones
algoritmos	Coincidencia con pandas, sorted y NumPy; estabilidad; no mutación	Secuencia vacía y de un elemento; bordes con valores repetidos	Largos distintos; k fuera de rango; mediana de vacío
benchmark	Exponente ≈ 1 para lineal y ≈ 2 para cuadrático	—	Menos de dos tamaños; tamaños iguales
dominio	Propiedades derivadas; orden natural; igualdad	Cambio de duración recalcula derivadas	Identidad de solo lectura; setters inválidos
metricas / jerarquia	Polimorfismo; totales recursivos; descenso por máximo	Nodo vacío; cache invalidada tras insertar	Cohorte vacía; percentil fuera de rango
preparacion / modelos	Media 0 y desviación 1; convergencia; pérdida decreciente	Columna constante; categoría no vista	Transformar sin ajustar; NaN; divergencia por tasa excesiva

Resultado de pytest tests/ -v

```

===== test session starts =====
===== 73 passed in 1.65s =====
(salida completa en el Anexo A)

```

Nivel 3: reglas de coherencia del dominio

El motor de reglas de la Fase 2 se reutiliza sin modificación sobre los artefactos de las Fases 3 y 4; es un caso de reutilización entre fases que la arquitectura modular hizo trivial.

Tabla 9. Reglas de validación y resultado

Regla	Comprobación	Estado
cohortes_y_arbol	La jerarquía contiene exactamente los titulados de la cohorte.	PASA
hojas_suman_total	Las hojas del árbol suman el total de la raíz (recursión correcta).	PASA
metrica_composite	Una métrica vía el árbol coincide con la misma vía la cohorte.	PASA
tasa_oportuna_coherente	La tasa de titulación oportuna sobre objetos coincide con la de F2.	PASA
convergencia_gd	El descenso de gradiente converge a la solución cerrada.	PASA
perdida_decrece	La pérdida final es menor que la inicial.	PASA
matrices_modelacion	Las matrices no tienen NaN y comparten columnas.	PASA
metricas_finitas	Todas las métricas de desempeño son números finitos.	PASA
r2_positivo	El modelo lineal explica varianza en el conjunto de prueba.	PASA

Nivel 4: ejecución de extremo a extremo

Los seis notebooks del proyecto se ejecutan de corrido con nbconvert en el entorno declarado, sin errores ni intervención manual. Las evidencias quedan versionadas en informe/evidencias_f4/ y en reports/, de modo que cada afirmación de este informe puede confirmarse en el repositorio.

4. Resultados

4.1 Panorama general

Tabla 10. Indicadores finales del estudio

Indicador	Valor	Lectura
Titulados de pregrado analizados	1.246.760	Período 2020-2025 más el parcial 2026-S1
Sobreduración media	2,76 sem.	El estudiante promedio cursa 2,76 semestres de más
Sobreduración mediana	1,0 sem.	La distancia con la media revela el peso de la cola
Índice de duración medio	1,43	Se cursa un 43 % más de tiempo del planificado
Titulación en el plazo teórico	19,0 %	Solo uno de cada cinco egresa en plazo
Rezago severo (más de 4 semestres)	20,4 %	Tantos como los que se titulan a tiempo
Titulación anticipada	4,1 %	Convalidaciones y cambios de carrera
Semestres-estudiante en exceso	3.551.990	Costo agregado del rezago en el período

El resultado central es inequívoco: cuatro de cada cinco titulados de pregrado en Chile egresan fuera del plazo que su propio plan de estudios declara. La distancia entre la media (2,76) y la mediana (1,0) anticipa lo que la Figura 1 confirma: la distribución es fuertemente asimétrica y una cola de rezago severo arrastra el promedio.

4.2 Distribución de la sobreduración

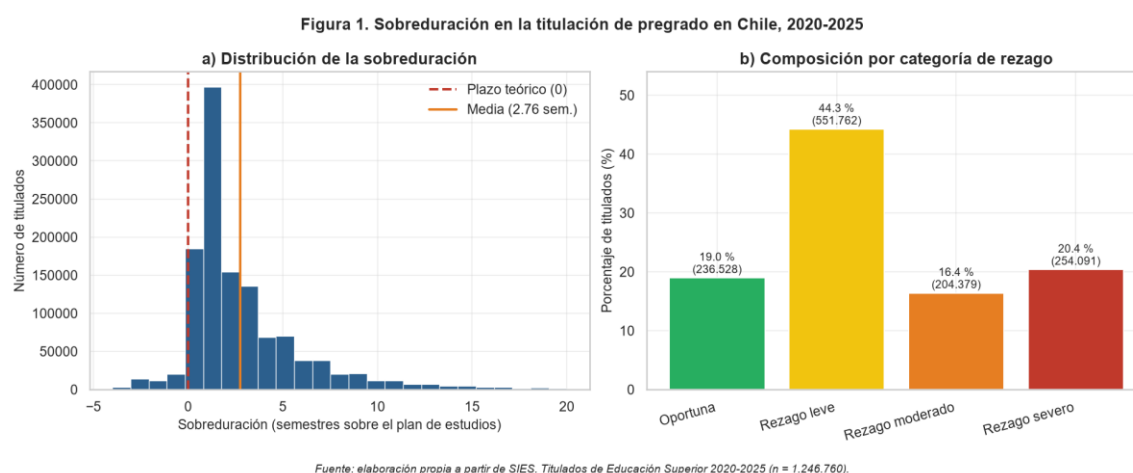


Figura 1. Distribución de la sobreduración y composición por categoría de rezago. Fuente: elaboración propia a partir de SIES, Titulados de Educación Superior 2020-2025.

La mayoría de los titulados se concentra en rezagos de uno a tres semestres, pero la cola derecha es larga y pesada. El rezago severo alcanza al 20,4 % —prácticamente la misma proporción que logra titularse en plazo (19,0 %)—, mientras que el rezago leve agrupa al 44,3 %. La lectura de política es directa: el rezago moderado es la norma, no la excepción, y la duración teórica declarada describe mal la trayectoria típica.

4.3 Factores asociados: área, institución y género

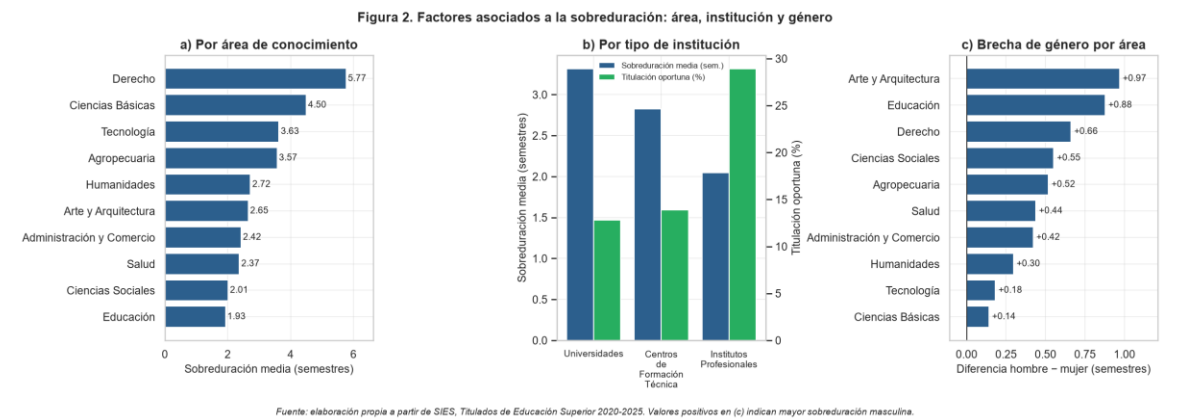


Figura 2. Sobreduración media por área de conocimiento, tipo de institución y brecha de género por área. Fuente: elaboración propia a partir de SIES, Titulados de Educación Superior 2020-2025.

Tabla 11. Sobreduración por área de conocimiento

Área de conocimiento	Titulados	Sobreduración media	Titulación oportuna
Derecho	28.748	5,77 sem.	8,7 %
Ciencias Básicas	13.505	4,50 sem.	6,3 %
Tecnología	317.385	3,63 sem.	14,1 %
Agropecuaria	30.915	3,57 sem.	10,7 %
Humanidades	7.813	2,72 sem.	15,9 %
Arte y Arquitectura	54.065	2,65 sem.	19,0 %
Administración y Comercio	285.186	2,42 sem.	21,7 %
Salud	232.468	2,37 sem.	21,9 %
Ciencias Sociales	129.637	2,01 sem.	21,8 %
Educación	147.038	1,93 sem.	22,4 %

Tabla 12. Sobreduración por tipo de institución y por género

Categoría	Titulados	Sobreduración media	Titulación oportuna
Universidades	569.497	3,32 sem.	12,8 %
Centros de Formación Técnica	214.592	2,83 sem.	13,9 %

Categoría	Titulados	Sobreduración media	Titulación oportuna
Institutos Profesionales	462.671	2,05 sem.	28,9 %
Hombre	542.694	3,29 sem.	16,0 %
Mujer	704.066	2,35 sem.	21,3 %

Los tres cortes muestran que el rezago no es homogéneo. Derecho (5,77 semestres) triplica a Educación (1,93): casi cuatro semestres —dos años— separan a la carrera más rezagada de la menos rezagada. Las universidades acumulan 3,32 semestres frente a 2,05 de los institutos profesionales, y su tasa de titulación oportuna es menos de la mitad, lo que sugiere que la duración declarada es menos realista en carreras universitarias largas.

La brecha de género es de 0,93 semestres a favor de las mujeres y se sostiene en las diez áreas de conocimiento sin excepción. Esa consistencia descarta que se explique por la composición de carreras: no es que las mujeres estudien carreras más cortas, es que dentro de cada área se titulan antes. Además, su tasa de titulación oportuna (21,3 %) supera a la masculina (16,0 %) en más de cinco puntos.

4.4 Modalidad y territorio

Tabla 13. Sobreduración por modalidad de enseñanza

Modalidad	Titulados	Sobreduración media
Presencial	1.126.649	2,86 sem.
No Presencial	97.184	1,84 sem.
Semipresencial	22.927	1,74 sem.

Tabla 14. Regiones con mayor y menor sobreduración (mínimo 5.000 titulados)

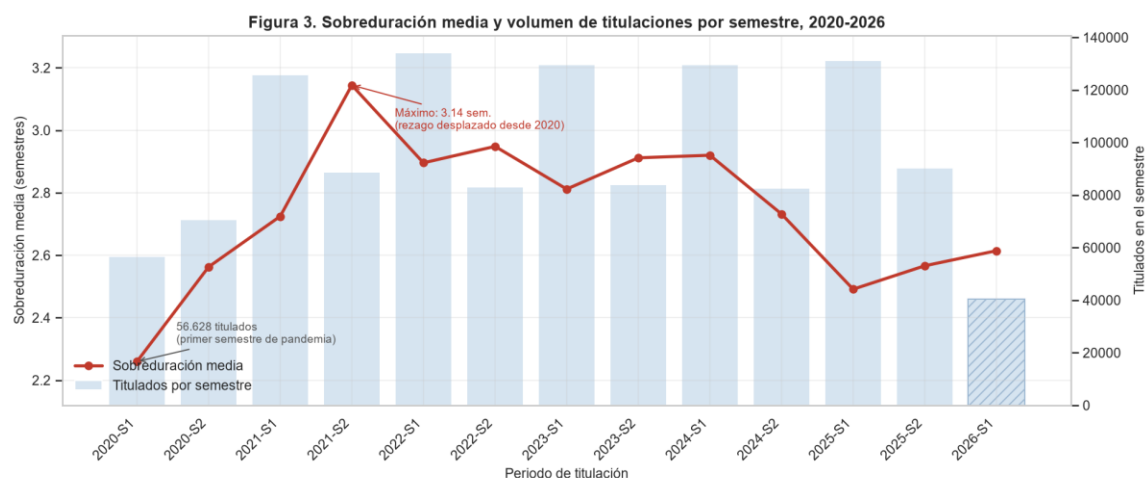
Región	Titulados	Sobreduración media
Atacama	14.809	3,69 sem.
Antofagasta	41.221	3,64 sem.
Arica y Parinacota	15.943	3,63 sem.
Tarapacá	19.979	3,53 sem.
...	...	...
Metropolitana	592.405	2,58 sem.
Lib. Gral. B. O'Higgins	32.583	2,54 sem.
Maule	62.321	2,43 sem.

La modalidad no presencial registra 1,84 semestres de sobreduración frente a 2,86 de la presencial. La diferencia debe leerse con cuidado: la modalidad a distancia atrae a un perfil

distinto —personas que trabajan, de mayor edad, con planes más flexibles— de modo que la comparación mezcla el efecto de la modalidad con el del perfil de quien la elige.

En el territorio, las regiones del norte encabezan el rezago (Atacama, 3,69 semestres) y las del centro-sur lo cierran (Maule, 2,43). La diferencia entre extremos supera un semestre completo.

4.5 Evolución temporal: el efecto de la pandemia



Fuente: elaboración propia a partir de SIES, Titulados de Educación Superior 2020-2025. La barra con trama corresponde a un semestre aún en curso.

Figura 3. Sobreduración media y volumen de titulaciones por semestre, 2020-2026. Fuente: elaboración propia a partir de SIES, Titulados de Educación Superior 2020-2025.

Tabla 15. Titulaciones y sobreduración media por semestre

Periodo	Titulados	Sobreduración media	Titulación oportuna
2020-S1	56.628	2,26 sem.	25,0 %
2020-S2	70.738	2,56 sem.	15,4 %
2021-S1	125.733	2,72 sem.	18,3 %
2021-S2	88.684	3,14 sem.	12,5 %
2022-S1	134.119	2,90 sem.	17,8 %
2022-S2	83.104	2,95 sem.	18,5 %
2023-S1	129.598	2,81 sem.	19,2 %
2023-S2	84.084	2,91 sem.	19,3 %
2024-S1	129.542	2,92 sem.	20,2 %
2024-S2	82.572	2,73 sem.	22,5 %
2025-S1	131.210	2,49 sem.	20,4 %
2025-S2	90.302	2,57 sem.	23,6 %

Periodo	Titulados	Sobreduración media	Titulación oportuna
2026-S1	40.446	2,61 sem.	9,9 %

La serie semestral revela un patrón que los promedios agregados ocultaban por completo. En 2020 se titularon 127.366 personas, unas 87.000 menos que en un año normal, y la sobreduración media tocó su mínimo (2,26 en 2020-S1). En 2021 el volumen se recuperó y la sobreduración alcanzó su máximo (3,14 en 2021-S2).

La lectura correcta no es que 2020 haya sido un buen año. Las titulaciones se postergaron: quienes debían egresar en 2020 lo hicieron en 2021 con uno o dos semestres adicionales encima. El mínimo de 2020 y el máximo de 2021 son dos caras del mismo fenómeno, y el mecanismo es de desplazamiento administrativo, no de deterioro académico. Desde 2024 la serie desciende de forma sostenida, un descenso que debe interpretarse a la luz de la limitación que documenta el apartado siguiente.

4.6 Sesgo de truncamiento por cohorte

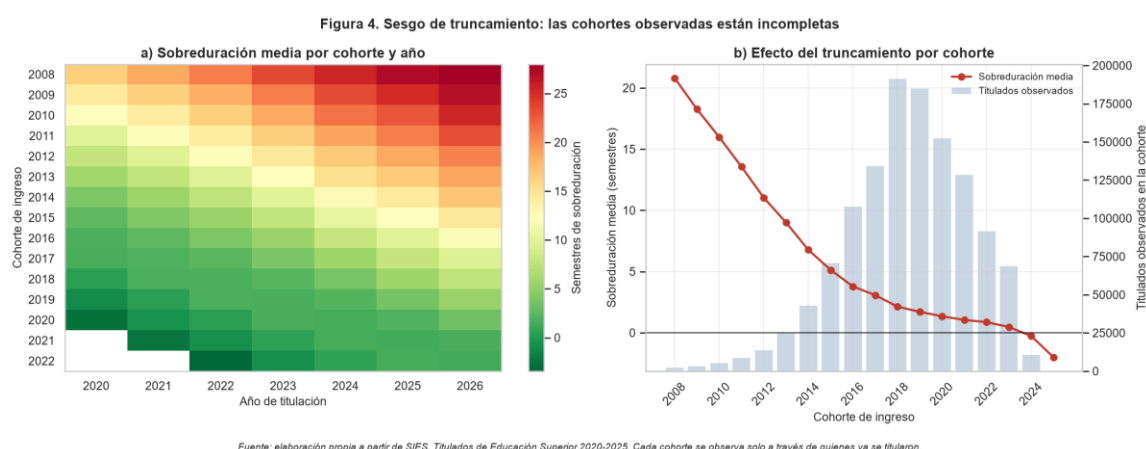


Figura 4. Sobreduración media por cohorte de ingreso y año de titulación, y efecto del truncamiento.
Fuente: elaboración propia a partir de SIES, Titulados de Educación Superior 2020-2025.

Tabla 16. Cohortes recientes: titulados observados y sobreduración media

Cohorte de ingreso	Titulados observados	Sobreduración media
2014	42.853	6,77 sem.
2015	70.711	5,13 sem.
2016	107.567	3,78 sem.
2017	134.348	3,08 sem.
2018	191.319	2,13 sem.
2019	184.973	1,71 sem.
2020	152.516	1,34 sem.

Cohorte de ingreso	Titulados observados	Sobreduración media
2021	128.566	1,05 sem.
2022	91.665	0,86 sem.
2023	68.826	0,47 sem.
2024	10.734	-0,27 sem.
2025	392	-1,99 sem.

La sobreduración media cae de forma monótona en las cohortes recientes hasta hacerse negativa. Sería un error celebrarlo como una mejora del sistema. De la cohorte 2024 solo se han titulado los excepcionalmente rápidos, que son los únicos que alcanzaron a hacerlo; de las cohortes antiguas, en cambio, quienes aparecen en una base de 2020-2025 son los extremadamente rezagados.

Ninguna cohorte está completa, y están incompletas en direcciones opuestas. El valor de una cohorte reciente subirá a medida que sus estudiantes más lentos se vayan titulando. Es una limitación inherente al diseño de la fuente, no un defecto del procesamiento, y condiciona toda lectura longitudinal: las comparaciones válidas son entre cohortes con horizonte de titulación equivalente.

4.7 Modelación desde primeros principios

Los modelos implementados en la Fase 3 —regresión lineal y logística por descenso de gradiente, sin bibliotecas de aprendizaje automático— se ejecutan sobre el conjunto analítico para cuantificar la asociación de cada factor. La regresión lineal se contrasta con su solución cerrada por ecuaciones normales, que es exacta, para verificar que la optimización converge al óptimo.

Tabla 17. Coeficientes de la regresión lineal y logística (variables estandarizadas)

Variable	Gradiente	Solución cerrada	Diferencia	Logística
dur_total_carr	-0,382	-0,382	0,000	0,055
edad_ingreso	-0,525	-0,525	0,000	0,291
genero=Mujer	-0,405	-0,405	0,000	0,128
tipo_inst_1=Institutos Profesionales	-0,544	-0,544	0,000	0,889
tipo_inst_1=Universidades	0,916	0,916	0,000	-0,040
area_conocimiento=Agropecuaria	0,712	0,712	0,000	-0,377
area_conocimiento=Arte y Arquitectura	0,002	0,002	0,000	0,109
area_conocimiento=Ciencias Básicas	1,358	1,364	0,006	-0,423

Variable	Gradiente	Solución cerrada	Diferencia	Logística
area_conocimiento=Ciencias Sociales	-0,522	-0,522	0,000	0,159
area_conocimiento=Derecho	2,862	2,862	0,000	-0,475
area_conocimiento=Educación	-0,802	-0,802	0,000	0,329
area_conocimiento=Humanidades	-0,543	-0,554	0,012	0,149
area_conocimiento=Salud	-0,377	-0,377	0,000	0,452
area_conocimiento=Tecnología	0,906	0,906	0,000	-0,304
modalidad=Presencial	-0,145	-0,145	0,000	-0,250
modalidad=Semipresencial	0,017	0,017	0,001	0,103

Tabla 18. Efecto del umbral de decisión en la regresión logística

Umbral	Exactitud	Precisión	Sensibilidad	F1
0,50	0,812	0,450	0,040	0,074
0,40	0,803	0,404	0,119	0,184
0,30	0,773	0,374	0,320	0,345
0,19	0,676	0,309	0,596	0,407

Dos lecturas, y la segunda importa tanto como la primera. El descenso de gradiente implementado a mano converge a la misma solución que las ecuaciones normales —la diferencia máxima entre coeficientes es despreciable—, lo que valida la implementación. Y los coeficientes son coherentes con el análisis descriptivo: estudiar en una universidad, cursar Derecho o ingresar con mayor edad se asocian a más sobreduración; ser mujer, a menos.

Pero el poder explicativo es bajo. Un R^2 de 0,085 en el conjunto de prueba significa que las variables institucionales y demográficas disponibles explican una fracción pequeña de la variación individual. En clasificación el resultado es aún más elocuente: con un 19 % de titulación oportuna, predecir siempre «no oportuna» acierta el 81,3 %, y el modelo con umbral 0,5 alcanza 81,2 %. No supera a la regla trivial. Bajar el umbral a la prevalencia (0,19) eleva la sensibilidad al 59,6 % a costa de la precisión, lo que convierte la elección del umbral en una decisión de política —a cuántos estudiantes acompañar— y no en un problema estadístico.

5. Discusión

5.1 Contraste con las hipótesis iniciales

La Fase 1 planteó tres hipótesis de trabajo. El análisis final permite evaluarlas con evidencia.

Tabla 19. Hipótesis iniciales frente a los resultados obtenidos

Hipótesis (Fase 1)	Evidencia (Fase 4)	Veredicto
La duración real supera sistemáticamente a la teórica	Sobreduración media de 2,76 semestres; solo el 19,0 % se titula en plazo	Confirmada
El rezago difiere por área de conocimiento y tipo de institución	Derecho triplica a Educación; las universidades superan a IP y CFT en más de un semestre	Confirmada
Las variables disponibles permitirían predecir el rezago individual	R ² de 0,085; la clasificación no supera la línea base	Refutada

La tercera hipótesis es la más valiosa precisamente porque no se cumplió. Muestra que el rezago es un fenómeno individual que los registros administrativos no capturan, y acota qué tipo de conclusiones puede sostener esta fuente. Un proyecto que solo confirmara lo que esperaba habría aprendido menos.

5.2 Limitaciones y riesgos

1. Sesgo de truncamiento. Ninguna cohorte está completa y las recientes están sesgadas hacia los estudiantes rápidos. Riesgo asociado: leer el descenso de la serie desde 2024 como una mejora del sistema. Mitigación adoptada: restringir las comparaciones longitudinales a cohortes con horizonte equivalente y documentar el sesgo explícitamente.
2. Sesgo de supervivencia. La base contiene únicamente a quienes se titularon. La deserción —probablemente la consecuencia más grave del rezago— es invisible. Riesgo: subestimar el problema, porque quien abandonó tras años de rezago no aparece.
3. Duración teórica declarada. La reporta cada institución y no está auditada. Un plan subdeclarado infla artificialmente la sobreduración medida de esa carrera.
4. Reconstrucción de la duración real. Supone continuidad de la trayectoria: no distingue entre un estudiante que cursó sin interrupciones y otro que congeló dos años. El indicador mide tiempo transcurrido, no tiempo cursado.

5. Variables ausentes. No hay información sobre situación laboral, financiamiento, salud ni carga familiar, que es justamente donde el modelo sugiere que reside la explicación. El bajo R^2 es evidencia de esa ausencia, no de un error de especificación.
6. Unidad de análisis. La fuente registra procesos de titulación, no personas: un estudiante con dos títulos aparece dos veces. Para las preguntas de este proyecto es la unidad correcta, pero impide leer los resultados como proporciones de personas.

5.3 Relación entre las fases y decisiones determinantes

Cada fase habilitó la siguiente, y algunas decisiones tempranas resultaron más determinantes de lo previsto.

- **F1 → F2.** Fijar las versiones de las dependencias y separar el código en módulos desde el inicio permitió que el pipeline creciera de 5 a 16 módulos sin reescrituras. Es la decisión de menor costo y mayor rendimiento del proyecto.
- **F2 → F3.** Disponer de un dataset validado hizo posible verificar cada algoritmo propio contra una referencia independiente sobre datos reales, y no sobre casos sintéticos. La corrección quedó establecida antes de medir la eficiencia.
- **F3 → F4.** El motor de reglas de la Fase 2 se reutilizó sin modificación sobre los artefactos de la Fase 3, y la fachada permitió que el notebook final coordine nueve módulos con cinco llamadas. La modularidad no fue un ejercicio académico: redujo trabajo real.
- **La decisión más costosa de revertir.** Marcar los atípicos en lugar de eliminarlos, en la Fase 2. Conservarlos permitió medir su efecto en la Fase 4 en vez de tener que rehacer la limpieza. La regla general que dejó el proyecto: cuando una decisión de limpieza es discutible, márcala y documéntala en vez de aplicarla de forma irreversible.

6. Conclusiones

6.1 Hallazgos sustantivos

1. Cuatro de cada cinco titulados de pregrado en Chile egresan fuera del plazo que su plan de estudios declara. El exceso medio es de 2,76 semestres, un 43 % sobre lo previsto, y suma 3.551.990 semestres-estudiante en el período analizado.
2. El rezago es estructural y no aleatorio. Derecho triplica a Educación, las universidades superan a institutos y centros de formación técnica, y las mujeres se titulan casi un semestre antes que los hombres de manera consistente en las diez áreas de conocimiento.
3. La pandemia desplazó titulaciones en lugar de empeorarlas. La caída de 2020 y el máximo de 2021 son el mismo fenómeno observado en dos momentos: un efecto de calendario administrativo, no de deterioro del desempeño académico.
4. Los registros administrativos no explican el rezago individual. Con un R^2 de 0,085 y una clasificación que no supera la línea base, el fenómeno responde a factores que la fuente no contiene. Esta conclusión negativa delimita el alcance legítimo de este tipo de análisis.

6.2 Aprendizajes técnicos

- **Verificar antes de optimizar.** Cada algoritmo propio se contrastó con una referencia independiente antes de medir su tiempo. Medir un algoritmo incorrecto no aporta información.
- **La cota teórica no basta.** El exponente empírico de quickselect (1,22 frente al 1,00 teórico) mostró que el análisis asintótico describe el crecimiento pero no la constante ni el costo de asignar memoria. La medición y la teoría son complementarias, no sustitutas.
- **Los invariantes pertenecen al objeto.** Trasladar las reglas del dominio a Titulado y Cohorte eliminó una clase entera de errores: un objeto no puede existir en estado incoherente.
- **Un fallo silencioso es peor que uno ruidoso.** Las tres mejoras de reproducibilidad aplicadas tras la Fase 2 comparten ese criterio, y las tres nacieron de un problema real detectado al reproducir el proyecto en un segundo equipo.

- **Los datos imponen sus propios límites.** El sesgo de truncamiento no se descubrió leyendo la documentación de la fuente, sino al cruzar cohorte de ingreso con año de titulación. Explorar los datos desde un ángulo nuevo reveló una limitación que ninguna fase anterior había detectado.

6.3 Mejoras propuestas, basadas en la evidencia del proyecto

Tabla 20. Mejoras priorizadas y evidencia que las motiva

Mejora	Evidencia que la motiva	Impacto esperado
Partición in situ (Hoare) en quickselect	Pico de 4.422 KB, el mayor del núcleo; exponente empírico 1,22 frente a 1,00 teórico	Menor consumo de memoria y escalamiento acorde a la cota
Versiones iterativas de merge sort y búsqueda binaria	Ambas dependen del límite de recursión de Python	Procesar el dataset completo sin reservas sobre la profundidad
Incorporar la base de matrícula además de la de titulados	El sesgo de supervivencia oculta la deserción	Estimar el rezago sin condicionar a haber egresado
Modelo de supervivencia en lugar de regresión lineal	El truncamiento por cohorte invalida la comparación longitudinal directa	Estimaciones válidas con datos censurados por la derecha
Perfilado por líneas además de timeit	Se sabe cuánto tarda cada función, pero no en qué línea se consume el tiempo	Optimizar con evidencia en lugar de intuición
Ampliar el modelo con interacciones y variables de trayectoria	R ² de 0,085 con las variables disponibles	Mayor poder explicativo, si se accede a nuevas fuentes

6.4 Comunicación de resultados

Los resultados se comunican en cuatro soportes complementarios, todos versionados en el repositorio:

- Los notebooks, para quien necesite auditar el procedimiento completo, celda por celda.
- Este informe, para la lectura estructurada del análisis y sus fundamentos.
- Un tablero interactivo, alimentado por scripts/exportar_powerbi.py, que permite a un tercero explorar los cortes por institución, área, territorio y cohorte sin ejecutar código. Incluye el score de riesgo del modelo logístico y un control de umbral que hace tangible el compromiso entre cobertura y capacidad de atención.

- Una presentación audiovisual de entre cinco y ocho minutos que sintetiza problema, metodología, resultados y conclusiones.

7. Trazabilidad de mejoras

Las mejoras aplicadas a lo largo del proyecto están registradas en CHANGELOG.md, en la raíz del repositorio, vinculadas al commit que las implementa. Cada fila puede verificarse con `git show` sobre el hash indicado.

Tabla 21. Mejoras aplicadas, commit de implementación e impacto técnico

Problema detectado	Mejora aplicada	Commit	Impacto
La normalización de texto recorría fila por fila 1,7 millones de cadenas	Se normaliza el catálogo de valores únicos y se mapea de vuelta	5e07669	Rendimiento: de 1,7 M de operaciones a unos 8 mil
Los tipos anulables de pandas rompían matplotlib y propagaban NA en comparaciones	Conversión explícita a float64 antes de graficar; NA tratados al marcar atípicos	f698c37	Corrección: 19 filas dejaban de marcarse en silencio
La celda de pruebas usaba el intérprete del kernel y fallaba sin mostrar la causa	Apunta al intérprete del entorno virtual e imprime stderr si falla	b8770f5	Reproducibilidad: fallo diagnosticable en vez de silencioso
Los notebooks dependían del Parquet completo, no versionado	Degradación automática a la muestra de 5.000 registros con aviso	40be4d9	Reproducibilidad: el proyecto corre sin datos externos
Los invariantes del dominio no estaban garantizados en ninguna parte	Modelo de dominio con validación en setters y derivadas de solo lectura	a1342a5	Modularidad: imposible construir un objeto incoherente
La agregación recorría la secuencia una vez por grupo, $O(n \cdot k)$	Agregación en una sola pasada $O(n)$ con dos acumuladores	8d87723	Rendimiento: 1,9× en tiempo y 860× en memoria
Los conteos por umbral recorrían la lista completa por categoría	Dos búsquedas binarias recursivas sobre una copia ordenada	8d87723	Rendimiento: 6.516× más rápido que el recorrido lineal
El descenso de gradiente devolvía pesos infinitos con tasas altas	Detección explícita de divergencia con excepción accionable	40be4d9	Robustez: error claro en vez de resultado inválido
El informe de la Fase 3 no seguía la estructura mínima exigida	Reorganización completa según la pauta de la asignatura	b5523ad	Documentación: correspondencia con los criterios de evaluación
Una cifra del informe comparaba contra la implementación equivocada	Verificación cruzada contra reports/tables y corrección del dato	12ae178	Confiabilidad: toda cifra rastreable a su archivo de origen

El criterio común a las diez mejoras es el mismo que atraviesa el proyecto: hacer explícito lo que fallaba en silencio, y dejar registro verificable de por qué se cambió.

8. Repositorio y notebooks

8.1 Estructura del repositorio

Organización del repositorio por fases

proyecto-titulados/		
-- F1/	F1_Definición.ipynb	Definición del problema y entorno
-- F2/	F2_1 · F2_2 · F2_3 .ipynb	Obtención, limpieza y validación
-- F3/	F3_Nucleo_Algoritmico.ipynb	Algoritmos, complejidad y POO
-- F4/	F4_Reporte_Analitico.ipynb	Reporte integrador (esta fase)
-- src/	16 módulos: pipeline de F2 y subpaquete núcleo/ de F3	
-- tests/	73 pruebas (22 del pipeline + 51 del núcleo)	
-- reports/	figures/ y tables/ con todas las salidas de los notebooks	
-- informe/	Los cuatro informes y sus evidencias por fase	
-- powerbi/	Guía del tablero de comunicación de resultados	
-- scripts/	exportar_powerbi.py	
-- docs/	Esquema oficial del SIES y mapa conceptual técnico	
-- data/	processed/ con la muestra versionada; raw/ e interim/ ignorados	
-- CHANGELOG.md	Trazabilidad de mejoras vinculada a commits	
-- requirements.txt	Dependencias con versión fijada	
-- README.md	Instalación, ejecución por fase y decisiones técnicas	

El README documenta la instalación del entorno, la ejecución de cada notebook con su tiempo estimado, las funciones del núcleo con su complejidad declarada y los criterios de optimización aplicados. El CHANGELOG registra la evolución del proyecto vinculada a commits verificables.

8.2 Notebooks del proyecto

Tabla 22. Notebooks, contenido y correspondencia con este informe

Notebook	Contenido	Apartado del informe
F1_Definición	Problema, entorno reproducible, esquema de datos	1, 2, 3.4
F2_1_Obtencion_Exploracion	Consolidación de los 6 CSV y exploración inicial	3.5
F2_2_Limpieza_Transformacion	Centinelas, duplicados y variables derivadas	3.5
F2_3_Validacion_Analisis	Reglas de validación y primeros resultados	3.8
F3_Nucleo_Algoritmico	Algoritmos, complejidad, POO y modelación	3.6, 3.7
F4_Reporte_Analitico	Integración F1-F4, figuras finales y discusión	4, 5, 6

Los seis notebooks se ejecutan de corrido sin errores. El de la Fase 4 no repite el trabajo de las anteriores: importa el pipeline de la Fase 2 y el núcleo de la Fase 3 a través de la fachada, de modo que ninguna lógica se redefine en celdas. Las tablas que produce se

exportan a reports/tables/f4_*.csv y las figuras a reports/figures/f4_*.png; todas las cifras de este informe provienen de esos archivos.

9. Referencias

Referencias en formato APA 7.ª edición, citadas en el cuerpo del informe y en los notebooks.

Bibliografía docente

Contreras Bravo, L. E., & Padilla Beltrán, J. E. (2024). Ciencia de datos con Python: transformación y selección de variables. Ediciones de la U.

Grus, J. (2019). Data science from scratch: First principles with Python (2.ª ed.). O'Reilly Media.

Wickham, H., & Grolemund, G. (2017). R para la ciencia de datos: importar, ordenar, transformar, visualizar y modelar datos. O'Reilly Media. <https://es.r4ds.hadley.nz/>

Fuente de datos

Servicio de Información de Educación Superior. (2025). Titulados de Educación Superior 2020-2025 [conjunto de datos]. Ministerio de Educación de Chile. <https://www.mifuturo.cl/bases-de-datos-de-titulados/>

Documentación técnica oficial

NumPy Developers. (2025). NumPy documentation (versión 2.5). <https://numpy.org/doc/stable/>

pandas Development Team. (2025). pandas documentation (versión 3.0). <https://pandas.pydata.org/docs/>

Python Software Foundation. (2025a). timeit — Measure execution time of small code snippets. <https://docs.python.org/3/library/timeit.html>

Python Software Foundation. (2025b). tracemalloc — Trace memory allocations. <https://docs.python.org/3/library/tracemalloc.html>

Python Software Foundation. (2025c). abc — Abstract base classes. <https://docs.python.org/3/library/abc.html>

Bibliografía complementaria

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>

McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3.^a ed.). O'Reilly Media.

Ramalho, L. (2022). Fluent Python: Clear, concise, and effective programming (2.^a ed.). O'Reilly Media.

Anexos

Anexo A. Salida completa de las pruebas automatizadas

```

===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Magister Ciencia de Datos\P...
cachedir: .pytest_cache
rootdir: D:\Magister Ciencia de Datos\Programacion para la Ciencia\Sumativas\proyecto-titulados
plugins: anyio-4.15.1
collecting ... collected 73 items
tests/test_nucleo.py::TestAgrupacion::test_media_por_grupo_coincide_con_pandas PASSED [ 1%]
tests/test_nucleo.py::TestAgrupacion::test_un_solo_grupo PASSED [ 2%]
tests/test_nucleo.py::TestAgrupacion::test_largos_distintos_lanzan_error PASSED [ 4%]
tests/test_nucleo.py::TestOrdenamiento::test_merge_sort_coincide_con_sorted PASSED [ 5%]
tests/test_nucleo.py::TestOrdenamiento::test_no_muta_la_entrada PASSED [ 6%]
tests/test_nucleo.py::TestOrdenamiento::test_vacia_y_un_elemento PASSED [ 8%]
tests/test_nucleo.py::TestOrdenamiento::test_es_estable PASSED [ 9%]
tests/test_nucleo.py::TestSeleccion::test_mediana_coincide_con_statistics PASSED [ 10%]
tests/test_nucleo.py::TestSeleccion::test_quickselect_recupera_cada_posicion PASSED [ 12%]
tests/test_nucleo.py::TestSeleccion::test_mediana_par_e_impar PASSED [ 13%]
tests/test_nucleo.py::TestSeleccion::test_k_fuera_de_rango_y_vacia PASSED [ 15%]
tests/test_nucleo.py::TestBusqueda::test_binaria_encuentra_y_no_encuentra PASSED [ 16%]
tests/test_nucleo.py::TestBusqueda::test_binaria_en_vacia PASSED [ 17%]
tests/test_nucleo.py::TestBusqueda::test_conteos_por_umbral_coinciden_con_recuento_directo PA...
tests/test_nucleo.py::TestBusqueda::test_cotas_en_bordes PASSED [ 20%]
tests/test_nucleo.py::TestBenchmark::test_medicion_basica PASSED [ 21%]
tests/test_nucleo.py::TestBenchmark::test_exponente_de_funcion_lineal_cercano_a_uno PASSED [ ...
tests/test_nucleo.py::TestBenchmark::test_exponente_cuadratico PASSED [ 24%]
tests/test_nucleo.py::TestBenchmark::test_estimar_orden_exige_dos_tamanos PASSED [ 26%]
tests/test_nucleo.py::TestBenchmark::test_comparar_ordena_por_tiempo PASSED [ 27%]
tests/test_nucleo.py::TestTitulado::test_propiedades_derivadas PASSED [ 28%]
tests/test_nucleo.py::TestTitulado::test_encapsulamiento_identidad_solo_lectura PASSED [ 30%]
tests/test_nucleo.py::TestTitulado::test_setters_validan PASSED [ 31%]
tests/test_nucleo.py::TestTitulado::test_orden_natural_e_igualdad PASSED [ 32%]
tests/test_nucleo.py::TestTitulado::test_desde_fila_con_dict_y_series PASSED [ 34%]
tests/test_nucleo.py::TestCohorte::test_protocolo_de_secuencia PASSED [ 35%]
tests/test_nucleo.py::TestCohorte::test_filtrar_y_agrupar_no_mutan PASSED [ 36%]
tests/test_nucleo.py::TestCohorte::test_strategy_con_metricas PASSED [ 38%]
tests/test_nucleo.py::TestCohorte::test_ordenados_con_merge_sort PASSED [ 39%]
tests/test_nucleo.py::TestCohorte::test_agregar_rechaza_tipos_ajenos PASSED [ 41%]
tests/test_nucleo.py::TestCohorte::test_metrica_en_cohorte_vacia_lanza_error PASSED [ 42%]
tests/test_nucleo.py::TestCohorte::test_desde_dataframe_exige_columnas PASSED [ 43%]
tests/test_nucleo.py::TestCohorte::test_percentil_valida_rango PASSED [ 45%]
tests/test_nucleo.py::TestJerarquia::test_totales_rekursivos_y_estructura PASSED [ 46%]
tests/test_nucleo.py::TestJerarquia::test_cache_se_invalida_al_insertar PASSED [ 47%]
tests/test_nucleo.py::TestJerarquia::test_metrica_composite_y_descenso PASSED [ 49%]
tests/test_nucleo.py::TestJerarquia::test_buscar_y_dataframe PASSED [ 50%]
tests/test_nucleo.py::TestJerarquia::test_nodo_vacio PASSED [ 52%]
tests/test_nucleo.py::TestPreparacion::test_escalador_estandar PASSED [ 53%]
tests/test_nucleo.py::TestPreparacion::test_estandar_invertir PASSED [ 54%]
tests/test_nucleo.py::TestPreparacion::test_minmax_en_cero_uno PASSED [ 56%]
tests/test_nucleo.py::TestPreparacion::test_transformar_sin_ajustar_y_dimensiones PASSED [ 57%]
tests/test_nucleo.py::TestPreparacion::test_onehot_omite_primera_y_categoria_desconocida PASS...
tests/test_nucleo.py::TestPreparacion::test_particion_reproducibile_y_disjunta PASSED [ 60%]

```

```

tests/test_nucleo.py::TestPreparacion::test_preparador_compone_y_exige_ajuste PASSED [ 61%]
tests/test_nucleo.py::TestModelos::test_descenso_converge_a_solucion_cerrada PASSED [ 63%]
tests/test_nucleo.py::TestModelos::test_perdida_decrece PASSED [ 64%]
tests/test_nucleo.py::TestModelos::test_logistica_separa_datos_separables PASSED [ 65%]
tests/test_nucleo.py::TestModelos::test_pesos_devueltos_son_copia PASSED [ 67%]
tests/test_nucleo.py::TestModelos::test_errores_controlados PASSED [ 68%]
tests/test_nucleo.py::TestModelos::test_metricas_clasificacion_extremos PASSED [ 69%]
tests/test_pipeline.py::test_duracion_real_caso_normal PASSED [ 71%]
tests/test_pipeline.py::test_sobreduracion_y_categoria PASSED [ 72%]
tests/test_pipeline.py::test_edad_descuenta_mes_no_cumplido PASSED [ 73%]
tests/test_pipeline.py::test_edad_cuando_aun_no_cumple_anios PASSED [ 75%]
tests/test_pipeline.py::test_semestre_de_titulacion_segun_mes PASSED [ 76%]
tests/test_pipeline.py::test_duracion_minima_es_uno PASSED [ 78%]
tests/test_pipeline.py::test_clasificacion_en_los_bordes PASSED [ 79%]
tests/test_pipeline.py::test_sentinelas_se_convierten_en_na PASSED [ 80%]
tests/test_pipeline.py::test_normalizacion_de_texto_respetar_nombres_propios PASSED [ 82%]
tests/test_pipeline.py::test_duplicados_sin_mrun_no_se_colapsan PASSED [ 83%]
tests/test_pipeline.py::test_duplicado_exacto_se_elimina PASSED [ 84%]
tests/test_pipeline.py::test_duracion_teorica_cero_no_divide_por_cero PASSED [ 86%]
tests/test_pipeline.py::test_registro_sin_fecha_nacimiento_se_marca_atipico PASSED [ 87%]
tests/test_pipeline.py::test_archivos_faltantes_lanza_excepcion PASSED [ 89%]
tests/test_pipeline.py::test_cargar_consolidado_inexistente PASSED [ 90%]
tests/test_pipeline.py::test_validador_detecta_dataset_corrupto PASSED [ 91%]
tests/test_pipeline.py::test_validador_aprueba_dataset_correcto PASSED [ 93%]
tests/test_pipeline.py::test_estado_antes_de_ejecutar_lanza_error PASSED [ 94%]
tests/test_pipeline.py::test_regla_con_excepcion_se_reporta_como_falla PASSED [ 95%]
tests/test_pipeline.py::test_tabla_agregada_respetar_minimo_de_casos PASSED [ 97%]
tests/test_pipeline.py::test_formato_numerico_chileno PASSED [ 98%]
tests/test_pipeline.py::test_configuracion_expone_versiones PASSED [100%]
===== 73 passed in 1.65s =====

```

Anexo B. Historial completo de commits

Hash	Fecha	Mensaje
12ae178	2026-09-27	feat(f4): notebook del reporte analítico integrador con cuatro figuras
7538f5c	2026-09-24	chore(f3): re-ejecuta los notebooks en el entorno local y actualiza salidas
b5523ad	2026-09-24	docs(f3): versión final del informe
b62d36a	2026-09-21	docs(f3): informe y evidencias alineados con el historial definitivo
9062995	2026-09-21	merge: integra la rama fase-3/nucleo-algoritmico en main
58cb930	2026-09-19	docs(f3): actualiza el informe y las evidencias con el historial definitivo
b769379	2026-09-19	docs(f3): agrega el PDF del informe con la fecha de entrega
d10b470	2026-09-19	docs(f3): Primer Informe F3
6d348f8	2026-09-19	docs(f3): informe técnico de la Fase 3 y evidencias de ejecución
b8770f5	2026-09-19	docs(f3): notebook del núcleo algorítmico ejecutado de corrido
d6df25e	2026-09-19	test(nucleo): 51 pruebas de casos normales, límite y excepciones
40be4d9	2026-09-19	feat(nucleo): preparación, modelos por descenso de gradiente y fachada
a1342a5	2026-09-19	feat(nucleo): modelo de dominio orientado a objetos

Hash	Fecha	Mensaje
8d87723	2026-09-19	feat(nucleo): algoritmos estructurados y recursivos con analisis de comp...
76867c7	2026-09-15	chore: prepara la rama de la Fase 3 y sanea el repositorio
807ad02	2026-09-09	Reejecucion de notebook en el equipo de trabajo con documento pdf
9d8c578	2026-09-09	Reejecucion de notebook en el equipo de trabajo con Jupyter LAB
75b8d4b	2026-09-09	Reejecucion de notebook en el equipo de trabajo
5d76ade	2026-09-08	docs: actualiza el informe y el notebook F1 tras corregir la autoria
004de50	2026-09-08	docs(informe): agrega el informe tecnico de las Fases 1 y 2
b70eaad	2026-09-08	style(viz): corrige la acentuacion de las etiquetas de los graficos
4418158	2026-09-08	style(viz): rotula las figuras con formato numerico chileno
b1021bc	2026-09-08	merge: integra la rama fase-2/pipeline-datos en main
fc75c8f	2026-09-08	docs(f2): ejecuta los notebooks y versiona figuras, tablas y muestra
5e07669	2026-09-08	perf(limpieza): normaliza el catalogo de categorias en vez de fila por fila
f698c37	2026-09-08	fix(viz,transformacion): compatibilidad con los tipos anulables de pandas
8334b7f	2026-09-08	docs(f2): notebooks de obtencion, limpieza, transformacion y validacion
d070aef	2026-09-08	docs(f1): notebook de definicion del proyecto y entorno reproducible
997c0d1	2026-09-08	feat(viz,pipeline): estilo unificado de figuras y orquestador ejecutable
6d2bb5f	2026-09-08	test: cubre casos normales, limite y excepciones del pipeline
2b8a0f2	2026-09-08	feat(validacion): motor de reglas orientado a objetos
a90d244	2026-09-08	feat(transformacion): reconstruye la duracion real y la sobreduracion
bf45180	2026-09-08	feat(limpieza): convierte codigos centinela del SIES en nulos explicitos
c8bb51e	2026-09-08	feat(ingesta): consolida los seis anios con lectura por bloques
02cd383	2026-09-08	feat(config): centraliza rutas, esquema de datos y parametros
8d219de	2026-09-08	chore: inicializa repositorio con entorno reproducible

Anexo C. Contribuciones y ramas

git shortlog -sn --all

33	Sebastian Antunez
3	santunez75

El proyecto es de autoría individual. Las dos identidades de Git corresponden a los dos equipos utilizados durante el desarrollo y ambas pertenecen al autor.